



INSTITUTO TECNOLÓGICO SUPERIOR DE LA
REGIÓN DE LOS LLANOS

INGENIERÍA MECATRÓNICA

Programacion Avanzada
Actividad: A3U2

Nombre:

Angel Daniel Cisneros Perez

Docente: Osbaldo Aragon Banderas
Fecha: 2026-06-22

ÍNDICE

1. Planteamiento del problema.....	2
2. Configuración en Proteus.....	2
3. Arquitectura del MLP.....	3
Funciones de activación.....	3
Codificación de entradas e interpretación de salidas.....	3
4. Obtención de pesos y bias.....	4
5. Pesos y bias finales.....	4
6. Interpretación de salidas y control de motores.....	5
7. Tabla de casos: sensores vs. acción.....	5
8. Validación en simulación.....	6
9. Código completo (Arduino).....	6
10. Repositorio y referencias.....	9
Referencias.....	9

1. Planteamiento del problema

Se implementa un perceptrón multicapa (MLP) que gobierna un robot diferencial TURTLE simulado en Proteus para seguir una línea. El robot percibe su entorno con tres sensores ópticos de línea (izquierdo, central y derecho) y actúa sobre dos motores de corriente directa (izquierdo y derecho). La red recibe el estado de los sensores y produce dos velocidades, una por motor, logrando un control diferencial: si la red acelera más un motor que el otro, el robot gira hacia el lado contrario y corrige su trayectoria.

Entradas: 3 señales digitales de los sensores de línea, codificadas como 0 (no detecta línea) o 1 (detecta línea).

Salidas: 2 valores continuos en el rango 0–1 (uno por motor) que se escalan a una señal PWM de 0 a 255 para fijar la velocidad de cada rueda.

2. Configuración en Proteus

El proyecto se arma con un microcontrolador Arduino UNO (ATmega328P) conectado al modelo de robot TURTLE de Proteus, descrito como un vehículo de tracción diferencial con detectores ópticos de línea. Los tres detectores del TURTLE se llevan a entradas del Arduino y las dos entradas de motor del TURTLE se controlan con salidas PWM. La siguiente asignación de pines es la usada en el firmware; debe coincidir con el cableado del esquema (puede ajustarse en las constantes del programa).

Señal	Pin Arduino	Descripción
Sensor izquierdo	A0	Entrada x[0] del MLP (1 = línea detectada)
Sensor central	A1	Entrada x[1] del MLP (1 = línea detectada)
Sensor derecho	A2	Entrada x[2] del MLP (1 = línea detectada)
Motor izquierdo	D5 (PWM)	Salida y[0] del MLP → velocidad (0–255)
Motor derecho	D6 (PWM)	Salida y[1] del MLP → velocidad (0–255)

Nota: el firmware (archivo PRUEBA_MPL.ino) se compila y se carga como archivo .hex en la propiedad Program File del componente Arduino dentro de Proteus.

3. Arquitectura del MLP

La red es un perceptrón multicapa totalmente conectado con la topología 3–4–2: una capa de entrada de 3 neuronas (los sensores), una capa oculta de 4 neuronas y una capa de salida de 2 neuronas (los motores).

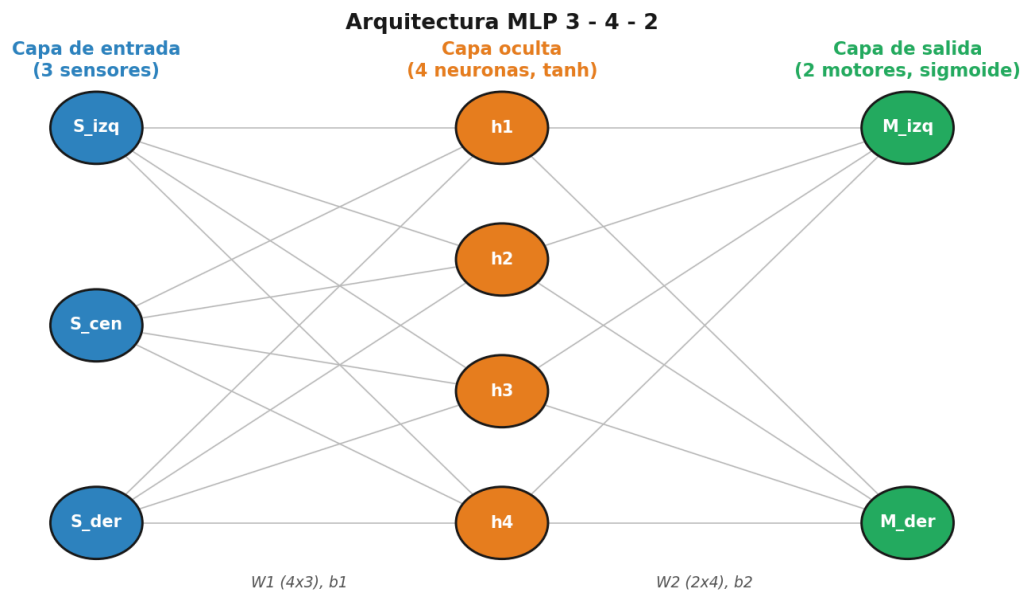


Figura 1. Arquitectura del MLP 3–4–2.

Funciones de activación

La capa oculta usa la tangente hiperbólica (tanh), que aporta la no linealidad necesaria para separar los ocho casos de sensores. La capa de salida usa la función sigmoide, que acota cada salida al intervalo 0–1, ideal para interpretarla como una fracción de velocidad y convertirla a PWM.

Codificación de entradas e interpretación de salidas

Las entradas son binarias (0/1) según cada sensor detecte o no la línea. Cada salida representa la velocidad relativa de un motor: $y[0]$ para el motor izquierdo y $y[1]$ para el derecho. Un valor cercano a 1 significa motor a alta velocidad y uno cercano a 0, motor casi detenido.

4. Obtención de pesos y bias

Los pesos (W) y bias (b) se obtuvieron entrenando la red fuera del microcontrolador, en Python con NumPy, mediante descenso de gradiente (retropropagación). El procedimiento fue:

- Conjunto de datos: las 8 combinaciones posibles de los 3 sensores (2^3) como entradas, y para cada una un par de velocidades objetivo (0–1) que produce la maniobra deseada (avanzar, girar a la izquierda o a la derecha).
- Modelo: capa oculta con tanh y capa de salida con sigmoide; función de pérdida de error cuadrático medio (MSE).
- Optimización: descenso de gradiente con tasa de aprendizaje 0.1 durante 20 000 épocas y semilla fija (seed = 0) para que los resultados sean reproducibles.
- Resultado: la red converge a un error muy bajo y se exportan $W1$, $b1$, $W2$ y $b2$ para copiarlos como constantes en el Arduino.

Tras el entrenamiento, el error cuadrático medio final fue $MSE \approx 0.000207$, lo que indica que las salidas predichas son prácticamente iguales a las deseadas.

5. Pesos y bias finales

Pesos y bias de la capa oculta (entrada $\rightarrow$ oculta):

Neurona oculta	$w \cdot S_{izq}$	$w \cdot S_{cen}$	$w \cdot S_{der}$	bias ($b1$)
h1	1.3297	2.0186	0.9993	-0.3493
h2	1.5907	0.9600	-1.1118	0.0083
h3	1.1926	0.1044	0.4579	-0.2100
h4	-0.4851	0.8251	1.4138	-0.2768

Pesos y bias de la capa de salida (oculta $\rightarrow$ salida):

Salida (motor)	$w \cdot h1$	$w \cdot h2$	$w \cdot h3$	$w \cdot h4$	bias ($b2$)
M_izq ($y[0]$)	1.1994	-1.1155	-1.1626	1.3913	0.5484
M_der ($y[1]$)	1.7629	1.1084	-0.2706	-1.7963	0.0440

6. Interpretación de salidas y control de motores

El control es diferencial. Cada salida de la red se convierte a PWM con la relación $PWM = \text{round}(y \times 255)$ y se envía al motor correspondiente con `analogWrite()`:

- Si el sensor derecho detecta la línea, la red sube la velocidad del motor izquierdo y baja la del derecho, por lo que el robot gira a la derecha hacia la línea.
- Si el sensor izquierdo la detecta, ocurre lo contrario y el robot gira a la izquierda.
- Si solo el sensor central la detecta (o ambos extremos), las dos salidas son similares y el robot avanza recto.
- Si ningún sensor detecta línea, ambas salidas quedan en torno a 0.5 (velocidad media), de modo que el robot sigue avanzando mientras vuelve a encontrarla.

7. Tabla de casos: sensores vs. acción

La siguiente tabla resume la respuesta del MLP ya entrenado para las 8 combinaciones de sensores (`y[0]`, `y[1]` son las salidas de la red; el PWM es el valor enviado a cada motor).

S_izq	S_cen	S_der	y[0]	y[1]	PWM izq	PWM der	Acción esperada
0	0	0	0.5004	0.5005	128	128	Avanza recto (busqueda: linea no detectada)
0	0	1	0.9516	0.2036	243	52	Giro a la derecha (corrige hacia la linea)
0	1	0	0.8387	0.8385	214	214	Avanza recto rapido (robot centrado)
0	1	1	0.9448	0.4536	241	116	Giro a la derecha (corrige hacia la linea)
1	0	0	0.2065	0.9659	53	246	Giro a la izquierda (corrige hacia la linea)
1	0	1	0.7225	0.7253	184	185	Avanza recto (ambos extremos / cruce)
1	1	0	0.4507	0.9286	115	237	Giro a la izquierda (corrige hacia la linea)
1	1	1	0.7192	0.7176	183	183	Avanza recto (ambos extremos / cruce)

8. Validación en simulación

En la simulación de Proteus el robot recorre la pista siguiendo la línea de forma estable: cuando el sensor central domina, avanza recto; cuando un sensor lateral se activa, la diferencia de PWM entre ruedas lo reorienta suavemente hacia la línea, recuperándose de pequeñas desviaciones. Los valores de la tabla de casos confirman este comportamiento antes de la prueba física en el simulador (por ejemplo, el caso 001 genera PWM 243/52, un giro marcado a la derecha, y el caso 100 genera 53/246, un giro a la izquierda).

Si en el escenario la polaridad de los sensores está invertida (1 = fondo, 0 = línea), basta con negar la lectura en la función leerSensores() para mantener la lógica.

9. Código completo (Arduino)

Archivo PRUEBA_MPL.ino. El Arduino únicamente ejecuta la propagación hacia adelante con los pesos y bias precargados; no realiza entrenamiento.

```
/*
 * =====
 * U2A3 - Perceptron Multicapa (MLP) para robot TURTLE
 * Seguidor de línea: 3 sensores de entrada -> 2 motores (diferencial)
 * -----
 * Arquitectura : 3 - 4 - 2
 *   - Entrada   : 3 neuronas (sensor izquierdo, central, derecho)
 *   - Oculta    : 4 neuronas, activacion tanh
 *   - Salida    : 2 neuronas, activacion sigmoide (0..1) -> PWM motores
 *
 * El Arduino SOLO ejecuta la propagacion hacia adelante (forward pass).
 * Los pesos y bias fueron entrenados FUERA del microcontrolador
 * (Python, descenso de gradiente, seed=0) y se cargan como constantes.
 *
 * Plataforma: Arduino UNO (ATmega328P) en Proteus, robot TURTLE.
 * Autor: Angel Daniel
 * =====
 */

#include <math.h>

// ----- Asignacion de pines -----
// Ajusta estos pines para que coincidan con tu cableado en Proteus.
const uint8_t PIN_SENSOR_IZQ = A0; // sensor de línea izquierdo
const uint8_t PIN_SENSOR_CEN = A1; // sensor de línea central
const uint8_t PIN_SENSOR_DER = A2; // sensor de línea derecho
const uint8_t PIN_MOTOR_IZQ  = 5;  // motor izquierdo (salida PWM)
const uint8_t PIN_MOTOR_DER  = 6;  // motor derecho  (salida PWM)

// ----- Topologia de la red -----
const uint8_t N_IN  = 3;
const uint8_t N_H   = 4;
const uint8_t N_OUT = 2;
```

```

// ----- Pesos y bias entrenados -----
// W1: pesos entrada -> oculta (N_H x N_IN)
const float W1[N_H][N_IN] = {
    {1.329676f, 2.018578f, 0.999286f},
    {1.590658f, 0.960025f, -1.111810f},
    {1.192569f, 0.104363f, 0.457913f},
    {-0.485088f, 0.825060f, 1.413771f}
};
// b1: bias de la capa oculta
const float b1[N_H] = { -0.349252f, 0.008270f, -0.210004f, -0.276794f };

// W2: pesos oculta -> salida (N_OUT x N_H)
const float W2[N_OUT][N_H] = {
    {1.199439f, -1.115519f, -1.162569f, 1.391347f},
    {1.762919f, 1.108431f, -0.270646f, -1.796346f}
};
// b2: bias de la capa de salida
const float b2[N_OUT] = { 0.548428f, 0.043954f };

// ----- Funciones de activacion -----
float activacionOculta(float z) {
    return tanhf(z); // tanh en la capa oculta
}

float activacionSalida(float z) {
    return 1.0f / (1.0f + expf(-z)); // sigmoide en la salida (0..1)
}

// ----- Lectura de sensores -----
// Devuelve 1 si el sensor detecta la linea, 0 en caso contrario.
// Si la polaridad esta invertida en tu escenario, niega la lectura.
void leerSensores(float x[N_IN]) {
    x[0] = (float) digitalRead(PIN_SENSOR_IZQ);
    x[1] = (float) digitalRead(PIN_SENSOR_CEN);
    x[2] = (float) digitalRead(PIN_SENSOR_DER);
}

// ----- Propagacion hacia adelante (inferencia) -----
void mlpForward(const float x[N_IN], float y[N_OUT]) {
    float a1[N_H];

    // Capa oculta: z = W1*x + b1 ; a1 = tanh(z)
    for (uint8_t j = 0; j < N_H; j++) {
        float z = b1[j];
        for (uint8_t i = 0; i < N_IN; i++) {
            z += W1[j][i] * x[i];
        }
        a1[j] = activacionOculta(z);
    }

    // Capa de salida: z = W2*a1 + b2 ; y = sigmoide(z)
    for (uint8_t k = 0; k < N_OUT; k++) {
        float z = b2[k];
        for (uint8_t j = 0; j < N_H; j++) {
            z += W2[k][j] * a1[j];
        }
        y[k] = activacionSalida(z);
    }
}

// ----- Mapeo salida -> PWM del motor -----
uint8_t aPWM(float salida) {
    float v = salida * 255.0f; // 0..1 -> 0..255
    return (uint8_t) constrain(v, 0, 255);
}

```



```

// ----- Configuracion -----
void setup() {
  pinMode(PIN_SENSOR_IZQ, INPUT);
  pinMode(PIN_SENSOR_CEN, INPUT);
  pinMode(PIN_SENSOR_DER, INPUT);
  pinMode(PIN_MOTOR_IZQ, OUTPUT);
  pinMode(PIN_MOTOR_DER, OUTPUT);
  Serial.begin(9600);
}

// ----- Bucle principal -----
void loop() {
  float x[N_IN];
  float y[N_OUT];

  leerSensores(x);          // 1) leer sensores
  mlpForward(x, y);         // 2) ejecutar el MLP (forward)

  // 3) interpretar salidas: y[0] -> motor izquierdo, y[1] -> motor derecho
  uint8_t pwmIzq = aPWM(y[0]);
  uint8_t pwmDer = aPWM(y[1]);

  // 4) controlar motores
  analogWrite(PIN_MOTOR_IZQ, pwmIzq);
  analogWrite(PIN_MOTOR_DER, pwmDer);

  // --- Depuracion por monitor serie ---
  Serial.print("Sensores ");
  Serial.print((int)x[0]); Serial.print((int)x[1]); Serial.print((int)x[2]);
  Serial.print(" -> PWM_izq=");
  Serial.print(pwmIzq);
  Serial.print(" PWM_der=");
  Serial.println(pwmDer);

  delay(20);
}

```

10. Repositorio y referencias

Repositorio GitHub (clase): https://github.com/danilopert-ui/U2A3_MLP_TURTLE/tree/main

Referencias

Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press.
<http://www.deeplearningbook.org>

Géron, A. (2019). Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow (2.^a ed.). O'Reilly Media.

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362.
<https://doi.org/10.1038/s41586-020-2649-2>

Labcenter Electronics. (s.f.). Proteus Design Suite: VSM for Arduino. Recuperado el 22 de junio de 2026, de <https://www.labcenter.com>

Arduino. (s.f.). Language reference (analogWrite, digitalRead). Recuperado el 22 de junio de 2026, de <https://www.arduino.cc/reference/en/>